ملاحظات الدرس — لك

# الاختبار بعقلية المهندس

البرمجة، المستوى ٤، الدرس ٤

المدة
٢٠ دقيقة

المواد
هاتف وعليه هذا الدرس بعد تنزيله، دفتر للاختبارات وملاحظات المراجعة

## أهداف التعلّم

- تكتب اختبارات وحدة بـ assert، ومنها الحالات الحدّية.
- تجد خطأً عند الحدود في البحث الثنائي وتصلحه.
- تعمل بالاختبار أولًا، وتراجع الكود مع الآخرين.

## النشاط: اكسر كودي

١. التمهيد (٣ دقائق): اكتب كل حالة حدّية تخطر لك لـ search(a, x). وقارن قائمتك بقائمة شريكك.
٢. التتبّع (٧ دقائق): تتبّع باليد البحث الخاطئ (while lo < hi) على search([5], 5). أين يخطئ؟
٣. الاختبار (٧ دقائق): اختر دالة جديدة، مثل median(values). واكتب أربعة أسطر assert، اثنان منها حالات حدّية، قبل أيّ كود.
٤. المراجعة (٣ دقائق): تبادل مع شريكك واكتب تعليقًا واحدًا على اختباراته: سؤالًا عن الكود، لا عن كاتبه أبدًا.

**تتبّع خطأ الحدود باليد هو لحظة الفهم: افعلها على الورق قبل تشغيل أيّ شيء.**

## للنقاش

- لماذا كان في ٩٠٪ من البحوث الثنائية التي كتبها محترفون أخطاء؟
- ماذا تثبت رؤية الاختبار يفشل أولًا؟
- كيف تجعل مراجعة الكود تبدو آمنة؟

## تحقّق من نفسك

- اكتب ثلاثة أسطر assert لحالات حدّية لدالة تعيد القيمة الوسطى في قائمة.

### جرّبها هذا الأسبوع

قبل أن تكتب دالتك القادمة، اكتب لها ثلاثة أسطر assert، أحدها عند حدّ. وشاهدها تفشل، ثم اجعلها تنجح.

ورقة عمل

# الاختبار بعقلية المهندس

البرمجة، المستوى ٤، الدرس ٤

الاسم _____ الفصل _____ التاريخ _____

**١ تعمل بالاختبار أولًا، وأضفت اختبارًا للشيء الصغير التالي، فنجح قبل أن تغيّر أيّ كود. ماذا يعني ذلك؟**

○ الاختبار لا يفحص شيئًا جديدًا، أو يفحص الشيء الخطأ ○ الدالة اكتملت ○ يمكنك الانتقال مباشرة إلى التحسين

**٢ يجد هذا البحث الثنائي هدفًا في منتصف القائمة تمامًا، لكنه يفشل عند الأطراف. أصلحه لتنجح كل اختبارات الحالات الحدّية.**

```
def search(a, x):
    lo, hi = 0, len(a) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

اكتب برنامجك:

**٣ يفشل assert top([-5, -2]) == -2: تعيد top القيمة 0. أيّ سطر فيه الخطأ؟**

ضع دائرة حول السطر الذي فيه الخطأ، ثم اكتبه صحيحًا:

```
1  def top(marks):
2      best = 0
3      for m in marks:
4          if m > best:
5              best = m
6      return best
```

**٤ رتّب دورة الاختبار أولًا.**

☐ اكتب أبسط كود ينجح فيه

☐ اكتب اختبارًا للشيء الصغير التالي، وشاهده يفشل

☐ كرّر مع الاختبار التالي

☐ نظّف الكود والاختبارات كلها ما زالت تنجح

**٥ أيّها حالات حدّية جيدة لـ search(a, x)؟**

☐ قائمة فيها عنصر واحد ☐ x هو العنصر الأول أو الأخير ☐ قائمة فارغة ☐ x في وسط قائمة طويلة

**✎ تذكّر دالة كتبتها ووثقت بها لأنها «بدت صحيحة». أيّ حالة حدّية قد تكسرها؟ اكتب سطر assert الذي يكشف ذلك.**

تحقّق من إجاباتك

# الاختبار بعقلية المهندس

البرمجة، المستوى ٤، الدرس ٤

**١** **الاختبار لا يفحص شيئًا جديدًا، أو يفحص الشيء الخطأ**

يجب أن يفشل الاختبار الجديد أولًا (أحمر): فهذا يثبت أنه قادر على كشف السلوك الناقص.

**٢**

```
def search(a, x):
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x:
            return mid
        elif a[mid] < x:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

تتخطّى hi < lo while آخر مكان باقٍ للبحث، حين يتساوى lo hig.

**٣** **السطر 2: `best = marks[0]`**

بدء best من 0 يفترض أن هناك دائمًا درجة فوق 0.

**٤** **من الأعلى إلى الأسفل: ٢، ١، ٤، ٣**

أحمر، أخضر، تحسين، ثم كرّر. ورؤية الاختبار يفشل أولًا تثبت أنه قادر على كشف المشكلة، والتنظيف عند الأخضر فقط يُبقي الكود آمنًا.

**٥** **قائمة فارغة، قائمة فيها عنصر واحد، x هو العنصر الأول أو الأخير**

الفارغة والوحيدة والأول والأخير هي حيث تخطئ حدود lo hig.